

Jetpack Compose第四章第一节教学讲义

(Compose动画核心用法)

各位同学，上一章我们学会了让组件响应手势，实现了“用户操作→组件响应”的交互。但如果只是简单的状态变化，比如点击后按钮直接变色、列表项突然消失，会显得非常生硬。优秀的APP都会用“动画”让状态变化更流畅自然——比如按钮点击时的缩放效果、页面跳转时的过渡动画。这节课咱们就聚焦Compose的动画能力，从最简单的数值变化到复杂的手势联动动画，一步步让你的组件“动”得优雅！

一、核心认知：Compose动画的本质

很多新手觉得动画很难，其实Compose的动画核心很简单：**在一段时间内，让组件的某个属性（比如大小、颜色、位置）从“初始值”平滑过渡到“目标值”**。就像汽车从静止（初始值）加速到100km/h（目标值），不是瞬间完成，而是有一个持续的加速过程。

Compose动画的优势

- **声明式API**：我们只需要告诉Compose“要做什么动画”，不用手动控制“每帧怎么变”，底层会自动计算中间状态；
- **与状态绑定**：动画和Compose的状态系统深度融合，状态变化时动画自动触发，无需手动启动；
- **轻量高效**：针对Compose的重组机制优化，避免不必要的性能消耗。

二、基础动画：简单值动画（最常用）

简单值动画是最基础的动画形式，用于让一个“单一数值”（比如尺寸、透明度、颜色数值）平滑变化。Compose提供了 `animateXxxAsState` 系列API，比如 `animateDpAsState`（尺寸动画）、`animateFloatAsState`（浮点值动画）、`animateColorAsState`（颜色动画）。

1. 尺寸动画：animateDpAsState

场景：点击按钮后，方块的大小从100dp平滑过渡到200dp，再次点击恢复。

实战1：可缩放的彩色方块

Code block

```
1 import androidx.compose.foundation.background
2 import androidx.compose.foundation.clickable
```

```

3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.size
5  import androidx.compose.runtime.Composable
6  import androidx.compose.runtime.getValue
7  import androidx.compose.runtime.mutableStateOf
8  import androidx.compose.runtime.remember
9  import androidx.compose.runtime.setValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.graphics.Color
12 import androidx.compose.ui.unit.dp
13 import androidx.compose.animation.animateDpAsState
14
15 @Composable
16 fun ScaleBoxDemo() {
17     // 状态：控制方块是否放大（true=放大，false=默认）
18     var isExpanded by remember { mutableStateOf(false) }
19
20     // 动画核心：根据状态自动计算中间尺寸
21     // 初始值100dp，目标值200dp（isExpanded为true时）
22     val boxSize by animateDpAsState(
23         targetValue = if (isExpanded) 200.dp else 100.dp,
24         label = "方块尺寸动画" // 标签：用于调试和性能分析
25     )
26
27     Box(
28         modifier = Modifier
29             .size(boxSize) // 绑定动画值
30             .background(Color.Blue)
31             .clickable { isExpanded = !isExpanded } // 点击切换状态
32     )
33 }

```

核心知识点

- `animateDpAsState`：接收“目标值”和“状态”，返回一个“可观察的动画值”——当状态变化时，这个值会从当前值平滑过渡到目标值；
- `label`：可选参数，用于给动画命名，方便在Android Studio的Profiler中调试动画性能；
- 动画触发：只要 `targetValue` 发生变化（这里由 `isExpanded` 控制），动画就会自动启动，无需调用“start”方法。

2. 颜色动画：animateColorAsState

场景：点击方块后，颜色从蓝色平滑过渡到红色，再次点击恢复。

实战2：颜色渐变的方块

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.clickable
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.size
5  import androidx.compose.runtime.Composable
6  import androidx.compose.runtime.getValue
7  import androidx.compose.runtime.mutableStateOf
8  import androidx.compose.runtime.remember
9  import androidx.compose.runtime.setValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.graphics.Color
12 import androidx.compose.ui.unit.dp
13 import androidx.compose.animation.animateColorAsState
14
15 @Composable
16 fun ColorChangeBoxDemo() {
17     var isRed by remember { mutableStateOf(false) }
18
19     // 颜色动画：从Blue过渡到Red
20     val boxColor by animateColorAsState(
21         targetValue = if (isRed) Color.Red else Color.Blue,
22         label = "方块颜色动画"
23     )
24
25     Box(
26         modifier = Modifier
27             .size(150.dp)
28             .background(boxColor) // 绑定动画颜色
29             .clickable { isRed = !isRed }
30     )
31 }
```

效果说明

点击方块后，颜色不会瞬间切换，而是会出现“蓝→紫→红”的渐变过程——这就是动画的“插值”效果，Compose会自动计算两个颜色之间的所有中间色。

三、常用场景动画：可见性与内容大小

除了简单的数值变化，开发中还经常需要“组件显示/隐藏”“内容大小变化”的动画。Compose提供了封装好的API，无需手动处理复杂的数值计算。

1. 可见性动画：AnimatedVisibility

场景：点击按钮后，文本从“滑入”显示；再次点击，文本“滑出”隐藏——替代生硬的“显示/消失”。

实战3：滑入滑出的文本

Code block

```
1  import androidx.compose.foundation.clickable
2  import androidx.compose.foundation.layout.Column
3  import androidx.compose.foundation.layout.padding
4  import androidx.compose.material3.Button
5  import androidx.compose.material3.Text
6  import androidx.compose.runtime.Composable
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.remember
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.unit.dp
13 import androidx.compose.animation.AnimatedVisibility
14 import androidx.compose.animation.slideInVertically
15 import androidx.compose.animation.slideOutVertically
16 import androidx.compose.animation.fadeIn
17 import androidx.compose.animation.fadeOut
18
19 @Composable
20 fun VisibilityAnimationDemo() {
21     // 状态：控制文本是否可见
22     var isVisible by remember { mutableStateOf(false) }
23
24     Column(
25         modifier = Modifier.padding(16.dp)
26     ) {
27         // 触发按钮
28         Button(onClick = { isVisible = !isVisible }) {
29             Text(text = if (isVisible) "隐藏文本" else "显示文本")
30         }
31
32         // 可见性动画组件
33         AnimatedVisibility(
34             visible = isVisible,
35             // 显示动画：从下方滑入+淡入
36             enter = slideInVertically(initialOffsetY = { it }) + fadeIn(),
37             // 隐藏动画：向下方滑出+淡出
38             exit = slideOutVertically(targetOffsetY = { it }) + fadeOut(),
39             modifier = Modifier.padding(top = 16.dp)
40         ) {
```

```

41         // 要执行动画的目标组件
42         Text(
43             text = "这是通过动画显示/隐藏的文本",
44             modifier = Modifier.padding(16.dp)
45         )
46     }
47 }
48 }

```

核心知识点

- `AnimatedVisibility`：包裹需要控制可见性的组件，通过 `visible` 状态控制显示/隐藏；
- `enter`：组件显示时的动画（这里用 `slideInVertically` 滑入+ `fadeIn` 淡入，用 `+` 组合多个动画）；
- `exit`：组件隐藏时的动画（这里用 `slideOutVertically` 滑出+ `fadeOut` 淡出）；
- `initialOffsetY = { it }`：`it` 代表组件自身的高度，意思是“从自身高度的位置开始滑入”（即从屏幕下方外滑入）。

2. 内容大小动画：animateContentSize

场景：点击按钮后，文本内容从“短文本”变为“长文本”，文本容器的大小会平滑过渡，而不是突然变大。

实战4：自适应大小的文本容器

Code block

```

1  import androidx.compose.foundation.border
2  import androidx.compose.foundation.clickable
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.material3.Text
6  import androidx.compose.runtime.Composable
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.remember
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.unit.dp
14 import androidx.compose.animation.animateContentSize
15
16 @Composable
17 fun ContentSizeAnimationDemo() {

```

```

18 // 状态：控制文本是否展开
19 var isExpanded by remember { mutableStateOf(false) }
20
21 // 文本内容：短文本/长文本切换
22 val textContent = if (isExpanded) {
23     "这是展开后的长文本内容，当点击文本容器时，容器会平滑地从短文本大小过渡到长文本大
    小，" +
24     "再次点击则收缩回短文本状态，整个过程由animateContentSize自动处理。"
25 } else {
26     "这是收缩后的短文本内容，点击展开。"
27 }
28
29 Column(modifier = Modifier.padding(16.dp)) {
30     // 带边框的文本容器（方便看到大小变化）
31     Text(
32         text = textContent,
33         modifier = Modifier
34             .padding(16.dp)
35             .border(1.dp, Color.Blue) // 蓝色边框
36             .padding(16.dp)
37             .clickable { isExpanded = !isExpanded }
38         // 核心：内容大小动画，自动过渡容器大小
39         .animateContentSize(
40             label = "内容大小动画"
41         )
42     )
43 }
44 }

```

核心优势

`animateContentSize` 是一个修饰符，直接作用于容器组件——当组件的内容发生变化（比如文本变长、子组件增减）时，它会自动计算容器的“初始大小”和“目标大小”，并实现平滑过渡，无需手动设置目标值。

四、进阶动画：多值动画与重复动画

当需要同时控制多个属性的动画（比如“大小+颜色+旋转同时变化”），或者需要动画无限重复（比如加载中的旋转图标）时，就需要用到多值动画和重复动画。

1. 多值动画：updateTransition

场景：点击方块后，同时实现“大小从100dp→200dp”“颜色从蓝→红”“旋转从0°→360°”三个动画效果。

实战5：多属性联动动画

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.clickable
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.size
5  import androidx.compose.runtime.Composable
6  import androidx.compose.runtime.getValue
7  import androidx.compose.runtime.mutableStateOf
8  import androidx.compose.runtime.remember
9  import androidx.compose.runtime.setValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.graphics.Color
12 import androidx.compose.ui.unit.dp
13 import androidx.compose.animation.core.updateTransition
14 import androidx.compose.animation.core.animateDp
15 import androidx.compose.animation.core.animateColor
16 import androidx.compose.animation.core.animateFloat
17 import androidx.compose.animation.core.tween
18
19 @Composable
20 fun MultiPropertyAnimationDemo() {
21     // 状态：用枚举表示不同的动画状态（Normal=正常，Expanded=展开）
22     enum class BoxState { Normal, Expanded }
23     var boxState by remember { mutableStateOf(BoxState.Normal) }
24
25     // 1. 创建过渡动画：管理多个属性的动画
26     val transition = updateTransition(
27         targetState = boxState,
28         label = "多属性动画"
29     )
30
31     // 2. 为每个属性定义动画
32     // 大小动画
33     val boxSize by transition.animateDp(
34         targetValueByState = { state ->
35             when (state) {
36                 BoxState.Normal -> 100.dp
37                 BoxState.Expanded -> 200.dp
38             }
39         },
40         label = "大小动画"
41     )
42
43     // 颜色动画
```

```

44     val boxColor by transition.animateColor(
45         targetValueByState = { state ->
46             when (state) {
47                 BoxState.Normal -> Color.Blue
48                 BoxState.Expanded -> Color.Red
49             }
50     },
51     label = "颜色动画"
52 )
53
54 // 旋转动画（浮点值，单位是弧度）
55 val boxRotation by transition.animateFloat(
56     targetValueByState = { state ->
57         when (state) {
58             BoxState.Normal -> 0f
59             BoxState.Expanded -> 360f * Math.PI.toFloat() / 180f // 转成弧度
60         }
61     },
62     label = "旋转动画",
63     animationSpec = tween(durationMillis = 1000) // 动画时长1秒
64 )
65
66 Box(
67     modifier = Modifier
68         .size(boxSize)
69         .background(boxColor)
70         .clickable {
71             // 点击切换状态 (Normal ↔ Expanded)
72             boxState = if (boxState == BoxState.Normal) BoxState.Expanded
73             else BoxState.Normal
74         }
75     // 应用旋转动画
76     .graphicsLayer(rotationZ = boxRotation * 180f / Math.PI.toFloat())
77 )

```

核心知识点

- `updateTransition`：管理“基于状态的多属性动画”，接收一个“目标状态”（这里是 `BoxState` 枚举），当目标状态变化时，所有关联的动画会同时启动；
- `targetValueByState`：为每个状态指定对应的属性值（比如 `BoxState.Expanded` 时，大小是 200dp，颜色是红色）；
- `animationSpec`：自定义动画效果，比如 `tween(durationMillis = 1000)` 表示动画时长1秒（默认动画是缓动效果，即先慢后快再慢）；

- `graphicsLayer(rotationZ)`：用于应用旋转效果，注意这里需要把弧度转成角度（因为rotationZ接收角度值）。

2. 重复动画：infiniteRepeatable

场景：实现“加载中”的旋转图标，动画无限重复，直到加载完成。

实战6：无限旋转的加载动画

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.layout.Box
3  import androidx.compose.foundation.layout.size
4  import androidx.compose.material3.CircularProgressIndicator
5  import androidx.compose.runtime.Composable
6  import androidx.compose.runtime.getValue
7  import androidx.compose.ui.Modifier
8  import androidx.compose.ui.graphics.Color
9  import androidx.compose.ui.unit.dp
10 import androidx.compose.animation.core.animateFloat
11 import androidx.compose.animation.core.infiniteRepeatable
12 import androidx.compose.animation.core.rememberInfiniteTransition
13 import androidx.compose.animation.core.tween
14
15 @Composable
16 fun LoadingAnimationDemo() {
17     // 1. 创建无限过渡动画（专门用于无限重复的动画）
18     val infiniteTransition = rememberInfiniteTransition(label = "加载动画")
19
20     // 2. 定义旋转动画（无限重复）
21     val rotation by infiniteTransition.animateFloat(
22         initialValue = 0f,
23         targetValue = 360f, // 目标旋转360度
24         animationSpec = infiniteRepeatable(
25             // 单次动画时长1秒
26             animation = tween(durationMillis = 1000)
27         ),
28         label = "旋转动画"
29     )
30
31     Box(
32         modifier = Modifier
33             .size(100.dp)
34             .background(Color.White)
35             .graphicsLayer(rotationZ = rotation) // 应用旋转
36     ) {
```

```

37         // 圆形进度条（作为加载图标）
38         CircularProgressIndicator(
39             color = Color.Blue,
40             modifier = Modifier
41                 .size(50.dp)
42                 .align(androidx.compose.ui.Alignment.Center)
43         )
44     }
45 }

```

核心知识点

- `rememberInfiniteTransition`：用于创建无限重复的动画，区别于普通的transition，它不需要外部状态触发，会一直重复；
- `infiniteRepeatable`：动画规范，设置动画无限重复；
- 停止动画：只需通过状态控制是否显示这个加载组件——当加载完成后，将组件设为不可见，动画自然停止。

五、高阶应用：手势与动画结合

动画的终极价值是“增强交互体验”，比如拖动组件时跟随手指，松开后组件平滑回到原位；或者滑动时组件的透明度随滑动距离变化。这就需要将上一章学的“手势”和本章的“动画”结合起来。

场景：拖动后回弹的方块

实现效果：按住方块拖动时，方块跟随手指移动；松开手指后，方块平滑回到屏幕中心。

实战7：手势联动动画

Code block

```

1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.gestures.detectDragGestures
3  import androidx.compose.foundation.layout.Box
4  import androidx.compose.foundation.layout.fillMaxSize
5  import androidx.compose.foundation.layout.size
6  import androidx.compose.runtime.Composable
7  import androidx.compose.runtime.getValue
8  import androidx.compose.runtime.mutableStateOf
9  import androidx.compose.runtime.remember
10 import androidx.compose.runtime.setValue
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.input.pointer.pointerInput

```

```

14 import androidx.compose.ui.unit.dp
15 import androidx.compose.animation.animateOffsetAsState
16 import androidx.compose.ui.geometry.Offset
17
18 @Composable
19 fun GestureAnimationDemo() {
20     // 屏幕中心位置 (假设屏幕宽度360dp, 高度640dp, 中心就是(180dp, 320dp))
21     val centerOffset = Offset(180.dp.toPx(), 320.dp.toPx())
22
23     // 状态1: 记录拖动时的实时位置 (用于跟随手指)
24     var dragOffset by remember { mutableStateOf(centerOffset) }
25
26     // 状态2: 控制是否启用回弹动画 (松开手指后为true)
27     var isAnimatingBack by remember { mutableStateOf(false) }
28
29     // 动画核心: 松开手指后, 从当前位置回弹到中心
30     val animatedOffset by animateOffsetAsState(
31         targetValue = if (isAnimatingBack) centerOffset else dragOffset,
32         label = "回弹动画",
33         // 动画结束后重置状态
34         finishedListener = { isAnimatingBack = false }
35     )
36
37     Box(modifier = Modifier.fillMaxSize()) {
38         Box(
39             modifier = Modifier
40                 .size(100.dp)
41                 .background(Color.Blue)
42                 // 应用动画位置
43                 .offset(
44                     x = animatedOffset.x.dp - 50.dp, // 减去方块一半宽度, 让中心对
45                     y = animatedOffset.y.dp - 50.dp
46                 )
47             // 拖动手势监听
48             .pointerInput(Unit) {
49                 detectDragGestures(
50                     // 手指按下时: 取消回弹动画, 跟随手指
51                     onDragStart = {
52                         isAnimatingBack = false
53                     },
54                     // 拖动过程中: 更新实时位置
55                     onDrag = { change, delta ->
56                         dragOffset = Offset(
57                             dragOffset.x + delta.x,
58                             dragOffset.y + delta.y
59                         )

```

```
60         change.consume()
61     },
62     // 手指松开时：启动回弹动画
63     onDragEnd = {
64         isAnimatingBack = true
65     }
66 )
67 }
68 )
69 }
70 }
```

核心逻辑

- 1. **拖动时**：通过 `onDrag` 回调实时更新 `dragOffset`，此时 `isAnimatingBack` 为false，动画的目标值就是 `dragOffset`，所以方块跟随手指移动；
- 2. **松开手指时**：设置 `isAnimatingBack` 为true，动画的目标值变为 `centerOffset`（屏幕中心），方块开始平滑回弹；
- 3. **offset计算**：因为 `offset` 修饰符是按组件左上角定位的，所以需要减去方块一半的宽度/高度，让方块的中心对齐到目标位置。

六、小节回顾：动画核心API速查表

动画类型	核心API	适用场景
简单值动画	animateDpAsState、 animateColorAsState、 animateFloatAsState	单一属性变化（大小、颜色、透明度）
可见性动画	AnimatedVisibility + enter/exit	组件显示/隐藏（滑入滑出、淡入淡出）
内容大小动画	Modifier.animateContentSize	内容变化导致容器大小变化（文本展开/收缩）
多值动画	updateTransition + animateXxx	多个属性同时变化（大小+颜色+旋转）
重复动画	rememberInfiniteTransition + infiniteRepeatable	加载动画、无限旋转/闪烁效果
手势联动动画	pointerInput + animateXxxAsState	拖动回弹、滑动渐变等交互动画

开发避坑指南

1. 动画与状态绑定：Compose动画的核心是“状态驱动”，不要手动控制动画帧，只需管理好目标状态；
2. 避免过度动画：不是所有状态变化都需要动画，过度动画会让APP显得臃肿，关键交互（如点击、切换）加动画即可；
3. 设置label参数：给每个动画添加label，方便后续调试动画性能问题；
4. 控制动画时长：普通状态变化动画时长建议200-300ms，加载动画时长建议1000ms左右，避免动画过慢影响体验。

七、课后作业：动手实现“交互型卡片”

需求：基于本节课知识点，实现一个支持“点击放大/缩小+拖动回弹+长按显示删除提示”的交互卡片，功能如下：

1. 基础功能：
默认状态：卡片大小150dp×100dp，背景色蓝色，显示“点击放大，长按删除”文本；
2. 点击卡片：平滑放大到200dp×150dp，背景色变为红色，文本变为“点击缩小”；再次点击平滑缩小回默认状态；
3. 进阶功能：
拖动卡片：按住卡片可拖动，跟随手指移动；松开手指后，卡片平滑回弹到屏幕中心；
4. 长按卡片：通过AnimatedVisibility显示“删除”按钮（滑入动画）；松开长按后，按钮滑出隐藏；
5. 点击删除按钮：卡片执行“缩小+淡出”动画后，从界面消失。
6. 技术要求：
用animateDpAsState实现大小动画，animateColorAsState实现颜色动画；
7. 用pointerInput监听点击、长按、拖动手势；
8. 用AnimatedVisibility实现删除按钮的显示/隐藏动画；
9. 用状态管理所有交互逻辑，避免硬编码。

提示：长按和点击的手势可以通过 `clickable` 的 `onLongClick` 参数实现，拖动则用 `detectDragGestures`。重点练习“多手势+多动画”的联动逻辑，确保不同交互下动画流畅自然，状态同步无误。

（注：文档部分内容可能由 AI 生成）